

LRU (LEAST RECENTLY USED) CACHE SYSTEM

GROUP MEMBERS

Name	ID	Department	Main Task
Omar Sayeed El-Said Abady	2403258221	AI(G1)	Report
Ziad Wahid Abdelaziz Radi	2403259500	AI(G1)	Task 1 && Task 2
Ahmed Sheriff Abdel Fattah	2403252673	AI(G1)	Task 3
Akram Mohamed Sayeed	2403254754	AI(G1)	Task 4
Ahmed Gamal Darwish	2403251323	AI(G1)	Task 5
Yehya Tamer Abdelhamid	2402241156	AI(G1)	Task 6
AlBadr Islam AlBadr Saad	2403259530	AI(G1)	Task 7
Hassan Ali Chit	2403251110	AI(G1)	Task 8
Gasem Mohamed Abdelaziz	2403257782	AI(G1)	Task 9
Omar Essam Mahmoud Ahmed	2403253520	AI(G1)	Task 10

OVERVIEW

1. Doubly Linked List

This class implements the doubly linked list used to maintain the usage order in the LRU Cache. The head represents the most recently used item, while the tail represents the least recently used item. The list is initialized empty by setting both head and tail to null.

The **insertAtFront()** method adds a node at the beginning of the list, which means the node becomes the most recently used item. The **removeNode()** method removes a given node from any position in the list while correctly updating the previous and next pointers. The **moveToFront()** method is used when an item is accessed, so it is moved to the front to mark it as recently used. Finally, **removeTail()** removes and returns the least recently used node, which is needed when the cache reaches its capacity and an old item must be evicted.

These operations support the LRU policy by keeping the most recently used items near the head and the least recently used item at the tail, as required in Tasks 2 to 6 of the project

2. Hash Table

The **hashTable<T>** class stores nodes using keys in an array.
It uses **GetHash** to find the correct index for each key.
Insert adds a node to the table.
Search finds a node by its key.
Remove deletes a node from the table.
Collisions are handled using **hashNext** chaining.
The average time complexity is $O(1)$, but the worst case is $O(n)$.

1 Task 1 (Node Design) :

```
//Task 1 (Node Design)
public class Node<T> {
    public int key;
    public T value;
    public Node<T> previous;
    public Node<T> next;
    public Node<T> hashNext;
    public Node(int key, T value){
        this.key = key;
        this.value = value;
        this.previous = null;
        this.next = null;
        this.hashNext = null;
    }
    public int getKey(){
        return key;
    }
    public T getValue(){
        return value;
    }
}
```

The **Node<T>** class represents a **generic node** used in more advanced data structures such as a **Doubly Linked List combined with a Hash Table** (e.g., LRU Cache).

It uses Java generics (<T>) to allow storing values of any data type, making the class flexible and reusable.

The class contains a key of type int and a value of type T, where T can represent any object type. It also includes previous and next references, which are used to connect nodes in a doubly linked list for bidirectional traversal.

Additionally, the **hashNext** reference is included to link nodes within the same hash bucket, forming a chain in case of hash collisions.

The constructor initializes the node with a given key and value, while setting all pointer

references to null.

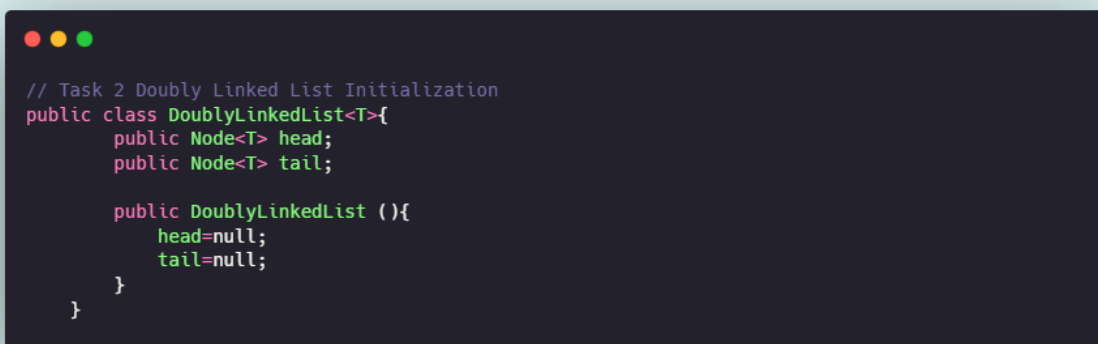
This ensures that the node is initially isolated and not connected to any other nodes.

The method **getkey()** returns the key stored in the node, allowing controlled access to this value.

Similarly, the method **getvalue()** returns the generic value stored in the node.

This design enables the node to be efficiently used in hybrid data structures that combine linked lists and hash-based storage.

2 Task 2 (Doubly Linked List Initialization) :



```
// Task 2 Doubly Linked List Initialization
public class DoublyLinkedList<T>{
    public Node<T> head;
    public Node<T> tail;

    public DoublyLinkedList (){
        head=null;
        tail=null;
    }
}
```

The **DoublyLinkedList** class represents a data structure that stores a collection of nodes connected in both directions.

It maintains two main references: head, which points to the first node, and tail, which points to the last node.

These references make it efficient to access and update elements at both the beginning and the end of the list.

The constructor initializes the list by setting both head and tail to null, indicating that the list is initially empty.

Each node in the list is linked to its previous and next nodes, allowing bidirectional traversal.

The head always points to the starting node, while the tail always points to the ending node.

When nodes are added or removed, these pointers are updated to maintain the structure of the

list.

This design improves performance in operations such as insertion and deletion compared to a singly linked list.

The structure allows easy navigation forward and backward through the elements.

Overall, the class serves as a container that organizes and manages nodes efficiently in a doubly linked format.

3 Task 3 (Insert at Front) :



```
// Task 3 Insert at Front
public void insertAtFront(Node<T> newNode) {
    newNode.previous = null;
    newNode.next = head;

    if (head != null) {
        head.previous = newNode;
    }
    head = newNode;
    if (tail == null) {
        tail = newNode;
    }
}
```

The **insertAtFront** method is used to add a new node at the beginning of the doubly linked list.

First, it sets both previous and next of the new node to null to ensure a clean state.

It then checks if the list is empty by verifying if head is null.

If the list is empty, both head and tail are assigned to the new node.

Otherwise, the new node is linked to the current head of the list.

The next of the new node is set to the current head.

The previous pointer of the old head is updated to point back to the new node.

After linking, the head reference is updated to the new node.

This guarantees that the new node becomes the first element in the list.

The method maintains the integrity of the doubly linked structure during insertion.

The **time complexity** of this operation is **O(1)** because it performs a fixed number of steps regardless of the size of the list.

4 Task 4 (Remove a Node) :

```
// Task 4 Remove a Node
public void removeNode(Node<T> node) {
    if (node == null)
        return;

    if (node == head) {
        head = node.next;
    } else {
        node.previous.next = node.next;
    }
    if (node == tail) {
        tail = node.previous;
    } else {
        node.next.previous = node.previous;
    }
    node.next = null;
    node.previous = null;
}
```

The **removeNode** method is used to delete a specific node from the doubly linked list. It first checks if the given node is null; if so, the method immediately returns without performing any operation.

If the node to be removed is the head, the head pointer is updated to the next node in the list. Otherwise, the previous node's next pointer is updated to skip the removed node and point to the next node.

The method then checks if the node is the tail of the list.

If it is the tail, the tail pointer is updated to the previous node.

Otherwise, the next node's previous pointer is updated to bypass the removed node.

After adjusting all links, the method sets both next and previous of the removed node to null.

This ensures the node is completely disconnected from the list and avoids potential issues like memory leaks.

The method maintains the correctness and integrity of the doubly linked list after deletion.

The **time complexity** of this operation is **O(1)** because the removal is done using direct references without any traversal.

5 Task 5 (Move Node to Front) :

```
// Task 5 Move Node to Front
public void moveToFront(Node<T> node) {
    if (node == head || node == null) {
        return;
    }
    removeNode(node);
    insertAtFront(node);
}
```

The **moveToFront** method is used to move a specific node to the beginning of the doubly linked list.

It first checks if the node is already the head or if the node is null.

If either condition is true, the method returns immediately because no action is needed.

If the node is not at the front, it is first removed from its current position in the list.

This is done by calling the `removeNode` method, which updates the surrounding links.

After removal, the node becomes isolated with no connections to the list.

The method then reinserts the same node at the front using the `insertAtFront` method.

This operation updates the necessary pointers to place the node as the new head.

By combining removal and insertion, the method ensures the list order is correctly updated.

This functionality is commonly used in structures like LRU Cache to mark recently used elements.

The **time complexity** of this operation is **O(1)** because both `removeNode` and `insertAtFront` run in constant time.

6 Task 6 (Remove Least Recently Used Item) :

```
// Task 6 Remove Least Recently Used Item
public Node<T> removeTail() {
    if (tail == null) {
        return null;
    }

    Node<T> removedNode = tail;
    if (head == tail) {
        head = null;
        tail = null;
    } else {
        tail = tail.previous;
        tail.next = null;
    }
    removedNode.next = null;
    removedNode.previous = null;
    return removedNode;
}
```

The **removeTail** method is used to remove and return the last node from the doubly linked list. It first checks if the tail is null, which means the list is empty, and returns null in that case. If the list is not empty, it stores the current tail node in a temporary variable called removedNode.

The method then checks if the list contains only one node by comparing head and tail. If both are equal, it means there is a single element, so both head and tail are set to null. Otherwise, the tail pointer is moved to the previous node in the list.

The new tail's next pointer is set to null to mark the end of the list.

After updating the list, the removed node's next and previous pointers are set to null.

This ensures the removed node is completely disconnected from the list structure.

Finally, the method returns the removed node for further use if needed.

The time complexity of this operation is **O(1)** because it directly accesses and updates pointers without any traversal.

7 Task 7 (Hash Function and Table Setup) :

```
//Task 7 Hash Function and Table Setup
public class hashTable<T> {
    Node<T>[] array_hash;
    int size;

    public hashTable(int size) {
        this.size = size;
        array_hash = (Node<T>[]) new Node[size];
    }

    public int GetHashCode(int key) {
        return Math.abs(key) % size;
    }
}
```

The **hashTable<T>** class represents a generic hash table that can store nodes with different value types.

It contains an array called **array_hash**, which is used to store references to Node<T> objects.

The variable **size** represents the total number of buckets available in the hash table.

The constructor receives the table size and assigns it to the **size** variable.

It then creates a new array of nodes with the given size.

The use of <T> makes the hash table generic and reusable for different data types.

The **GetHash** method is responsible for converting a key into a valid array index.

It uses **Math.abs(key)** to avoid negative index values.

The modulo operator **% size** ensures that the generated index stays within the array range.

The time complexity of the **GetHash** method is **O(1)** because it performs a fixed number of operations.

8

Task 8 (Hash Table Operations):

```
//Task 8 Hash Table Operations
public void Insert(int key, Node<T> node) {
    int index = GetHash(key);
    node.hashNext = array_hash[index];
    array_hash[index] = node;
}

public Node<T> Search(int key) {
    int index = GetHash(key);
    Node<T> current = array_hash[index];
    while (current != null) {
        if (current.key == key) {
            return current;
        }
        current = current.hashNext;
    }
    return null;
}

public void Remove(int key) {
    int index = GetHash(key);
    Node<T> current = array_hash[index];
    Node<T> previous = null;
    while (current != null) {
        if (current.key == key) {
            if (previous == null) {
                array_hash[index] = current.hashNext;
            } else {
                previous.hashNext = current.hashNext;
            }
            return;
        }
        previous = current;
        current = current.hashNext;
    }
}
```

The **Insert method** adds a new node into the hash table using its key.

It first calculates the correct array index by calling the **GetHash** method.

The new node is inserted at the beginning of the linked chain in that bucket.

The **hashNext** pointer is used to handle collisions between nodes with the same hash index.

The **Search method** looks for a node by calculating its index first.

It then traverses the linked chain in that bucket until it finds the required key.

If the key is found, the method returns the matching node.

If the key does not exist, the method returns null.

The **Remove method** deletes a node by updating the links between nodes in the same bucket.

The average time complexity for Insert, Search, and Remove is **O(1)**, while the worst case is **O(n)** when many collisions occur.

9

Task 9 (LRU Cache Core Logic):

```

//Task 9 LRU Cache Core Logic
public class LRUCache<T> {
    private DoublyLinkedList<T> list;
    private hashTable<T> hashTable;
    private int capacity;
    private int size;
    public LRUCache(int capacity) {
        this.capacity = capacity;
        this.list = new DoublyLinkedList<>();
        this.hashTable = new hashTable<>(capacity * 2);
    }
    public T get(int key) {
        Node<T> node = hashTable.Search(key);
        if (node == null) {
            return null;
        }
        list.moveToFront(node);
        return node.value;
    }
    public void put(int key, T value) {
        Node<T> existing = hashTable.Search(key);
        if (existing != null) {
            existing.value = value;
            list.moveToFront(existing);
            return;
        }
        if (size >= capacity) {
            Node<T> tail = list.removeTail();
            if (tail != null) {
                hashTable.Remove(tail.key);
                size--;
            }
        }
        Node<T> newNode = new Node<>(key, value);
        list.insertAtFront(newNode);
        hashTable.Insert(key, newNode);
        size++;
    }
    public void remove(int key) {
        Node<T> node = hashTable.Search(key);
        if (node == null)
            return;
        list.removeNode(node);
        hashTable.Remove(key);
        size--;
    }
    public void display() {
        list.display();
    }
}

```

The **LRUCache<T>** class implements a generic Least Recently Used cache.

It uses a **DoublyLinkedList** to keep track of the usage order of elements.

It also uses a **hashTable** to access nodes quickly using their keys.

The capacity variable stores the maximum number of items allowed in the cache.

The get method searches for a key and moves the found node to the front because it becomes recently used.

If the key is not found, the get method returns null.

The put method updates the value if the key already exists and moves it to the front.

If the cache is full, it removes the least recently used node from the tail.

The remove method deletes a node from both the linked list and the hash table.

The average time complexity for get, put, and remove is **$O(1)$** because the cache uses direct hashing and linked list operations.

10 Task 10 (Capacity Handling and Testing):

```
//Task 10 Capacity Handling and Testing
public class TestingLRU {
    public static void main(String[] args) {
        System.out.println("==== LRU CACHE TEST =====\n");

        System.out.println("Test1: Capacity Rule");
        LRUcache<Integer> cache1 = new LRUcache<>(3);
        cache1.put(1, 100);
        cache1.put(2, 200);
        cache1.put(3, 300);
        cache1.put(4, 400);
        cache1.display();

        System.out.println("Test2: Insertion & Retrieval");
        LRUcache<Integer> cache2 = new LRUcache<>(3);
        cache2.put(1, 100);
        cache2.put(2, 200);
        cache2.put(3, 300);
        System.out.println(cache2.get(1));
        System.out.println(cache2.get(2));
        System.out.println(cache2.get(3));
        cache2.display();

        System.out.println("Test3: Update Behavior");
        LRUcache<Integer> cache3 = new LRUcache<>(3);
        cache3.put(1, 100);
        cache3.put(2, 200);
        cache3.put(3, 300);
        cache3.put(2, 999);
        System.out.println(cache3.get(2));
        cache3.display();

        System.out.println("Test4: Eviction Order");
        LRUcache<Integer> cache4 = new LRUcache<>(3);
        cache4.put(1, 100);
        cache4.put(2, 200);
        cache4.put(3, 300);
        cache4.get(1);
        cache4.put(4, 400);
        cache4.display();

        System.out.println("Test5: Edge Cases");
        System.out.println("Empty Cache Test");
        LRUcache<Integer> emptyCache = new LRUcache<>(2);
        System.out.println(emptyCache.get(10));
        emptyCache.remove(10);
        emptyCache.display();

        System.out.println("Repeated Access Test");
        LRUcache<Integer> cache5 = new LRUcache<>(3);
        cache5.put(1, 100);
        cache5.put(2, 200);
        cache5.put(3, 300);
        cache5.get(2);
        cache5.get(2);
        cache5.get(2);
        cache5.display();
    }
}
```

The **TestingLRU** class is used to test the functionality of the LRU Cache. It creates different cache objects with fixed capacities to check how the cache behaves.

Test1 checks the capacity rule by inserting more elements than the allowed capacity.

Test2 tests insertion and retrieval using the put and get methods.

Test3 checks update behavior by changing the value of an existing key.

Test4 tests eviction order by accessing an item before adding a new one.

This ensures that the least recently used item is removed first.

Test5 handles edge cases such as searching in an empty cache and removing a non-existing key.

It also tests repeated access to the same key to make sure it stays recently used.

Overall, this class verifies that insertion, retrieval, updating, removal, and capacity handling work correctly.

11 OUT PUT:

```
===== LRU CACHE TEST =====  
  
Test1: Capacity Rule  
4:4003:3002:200  
Test2: Insertion & Retrieval  
100  
200  
300  
3:3002:2001:100  
Test3: Update Behavior  
999  
2:9993:3001:100  
Test4: Eviction Order  
4:4001:1003:300  
Test5: Edge Cases  
Empty Cache Test  
null  
  
Repeated Access Test  
2:2003:3001:100
```